INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES STRASBOURG

PI — INSA Rouen Normandie Département ISIA

PI SGBD

Projet Intégratif développement orienté objet



Maïssa BEN SETHOUM

Yesmina ELLOUMI

Ewen GUICHARD

Youssef HAJBI

Jules ROUDIL

3 juin 2026

Table des matières

1 Modelisation **3**
1.1 Diagramme de Cas d'Utilisation 3
1.2 Modélisation Conceptuelle du SGBD 3
1.2.1 Description des entités du domaine 3
1.2.2 Modélisation du moteur d'évaluation logique 4
1.2.3 Justification des relations du modèle 4

2 Conception Détaillée du Système **5**
2.1 Architecture Globale du SGBD et Encapsulation 5
2.2 Typage Fort et Polymorphisme des Données 5
2.3 Moteur d'Évaluation Logique 6
2.4 Découplage pour l'Algèbre Relationnelle (SELECT) 6

3 Implementation **7**
3.1 Implémentation de l'Interpréteur de Commandes SQL 7
3.1.1 Configuration globale et analyse lexicale 7
3.1.2 Architecture et liaison avec le SGBD 7
3.1.3 Gestion avancée du cycle de vie des requêtes 7
3.2 Implémentation du Moteur SGBD en Java 8
3.2.1 Typage fort et interface `Valeur` 8
3.2.2 Gestion des alias avec `TableSource` 9
3.2.3 Évaluation des conditions 9

Conclusion **10**

Introduction

En tant que membres de l'équipe technique de la Batcave, nous faisons face à un défi de taille : Gotham City souffre d'une insécurité numérique grandissante. Pour y remédier et centraliser les données stratégiques du Chevalier Noir, notre mission est de concevoir un système de gestion de base de données (SGBD) sur mesure, baptisé le *Student Gotham Bat Database*.

Batman nous a fourni un cahier des charges très strict, reflétant ses exigences opérationnelles et son attachement à une syntaxe SQL précise et rigoureuse. L'objectif ultime de ce projet intégratif est de développer notre propre moteur de base de données de manière totalement autonome, depuis l'interpréteur de commandes textuelles jusqu'au stockage physique des données.

Ce rapport documente notre démarche d'ingénierie et détaille les raisonnements algorithmiques ainsi que les choix de conception orientée objet qui nous ont guidés tout au long du développement.

Pour suivre la progression de notre travail, le document est structuré en trois grandes parties :
La modélisation : où nous présentons la vision métier du système à l'aide des diagrammes de cas d'utilisation et conceptuels, posant ainsi les bases du SGBD.
La conception : dédiée à l'architecture détaillée du moteur et à la dynamique des interactions entre nos objets grâce au diagramme de classe.
L'implémentation : axée sur le code Java concret et la configuration de l'interpréteur avec JavaCC, permettant de traduire les commandes textuelles en actions physiques sur la base de données.

1 Modelisation

1.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation synthétise les fonctionnalités offertes par le SGBD à l'utilisateur final. Les fonctionnalités sont structurées en trois grands ensembles cohérents avec la grammaire :

- **Définition des Données** : L'utilisateur peut créer de nouvelles structures de tables en spécifiant le schéma attendu, ou détruire définitivement des tables existantes.
- **Manipulation des Données** : Le système permet d'alimenter les tables, de purger des données sous condition et de requêter la base.
- **Gestion Système** : L'utilisateur dispose de commandes utilitaires pour administrer sa session, comme la consultation du catalogue des tables ou l'arrêt du programme.

Une relation d'inclusion (*include*) est modélisée entre l'action de quitter l'application et la sauvegarde des données. Ce choix de conception garantit la persistance des données : l'utilisateur n'a pas besoin de lancer manuellement une commande d'écriture, le SGBD sérialise automatiquement l'état des tables sur le disque avant de couper le flux.



FIGURE 1 – Diagramme de Cas d'Utilisation.

1.2 Modélisation Conceptuelle du SGBD

La phase de modélisation a pour objectif de dégager les concepts clés et les règles métier de notre système de gestion de bases de données, indépendamment des choix techniques d'implémentation sous-jacents. Le système est conçu autour de la structure relationnelle classique, adaptée aux spécificités de la grammaire *Gotham Bat Database*.

1.2.1 Description des entités du domaine

Le modèle s'articule autour des entités fondamentales suivantes :

- **SGBD** : L'entité maîtresse qui orchestre la base de données. Elle centralise les opérations de manipulation et fait le lien avec l'interpréteur de commandes.
- **Table** : Représente une structure de données nommée. Conceptuellement, une table est l'association d'un schéma (un ensemble d'attributs) et d'un ensemble de données (les lignes).
- **Attribut** : Définit une colonne de la table par un nom unique et un type de données restrictif parmi ceux autorisés par notre grammaire (`INT`, `VARCHAR`, `SERIAL`).
- **Ligne et Valeur** : Une `Ligne` matérialise un enregistrement concret dans la table. Elle contient un ensemble de structures `Valeur` qui encapsulent la donnée réelle tout en respectant le type de l'attribut correspondant.

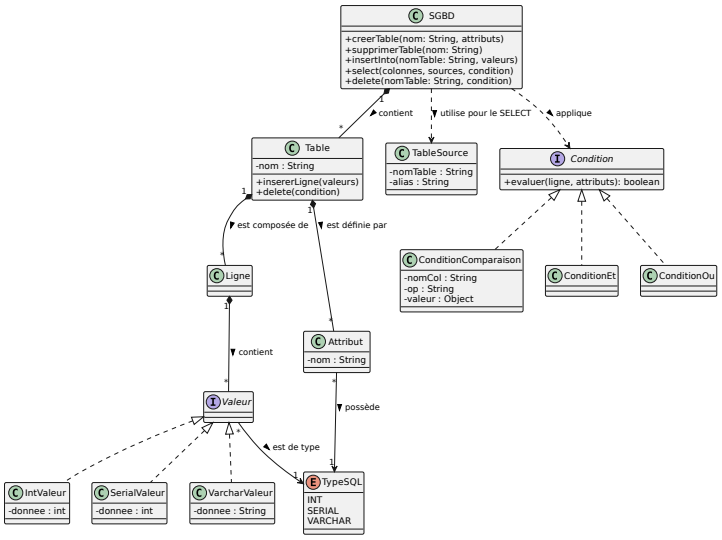


FIGURE 2 – Diagramme de Modélisation.

1.2.2 Modélisation du moteur d'évaluation logique

Un point clé du modèle réside dans la gestion des filtres introduits par la clause `WHERE`. Pour modéliser la combinatoire des expressions logiques de manière évolutive, nous avons opté pour la création de :

- L'interface **Condition** définit un contrat générique d'évaluation sur une ligne donnée.
- Une **ConditionComparaison** effectuant une comparaison stricte (`=`, `>`, `<`) entre une colonne et une valeur (ou une autre colonne).
- Les classes **ConditionEt** et **ConditionOu** qui se composent de deux sous-conditions (gauche et droite) et permettent de lier récursivement un nombre infini de critères logiques.

1.2.3 Justification des relations du modèle

Les choix des relations du diagramme traduisent les contraintes de cycle de vie de nos objets :

- Les liens entre `SGBD` → `Table` → `Ligne`/`Attribut` sont modélisés par des **compositions** (losanges pleins). Cela indique une dépendance existentielle forte : si une table est supprimée du SGBD, ses lignes et ses attributs cessent d'exister.
- Les types de données (`TypeSQL`) sont partagés via des associations simples avec les `Attributs` et les `Valeurs` pour garantir la cohérence sémantique lors des vérifications de formats à l'insertion.

2 Conception Détaillée du Système

La phase de conception consiste à traduire les besoins de la grammaire SQL et les concepts de modélisation en une architecture logicielle robuste, extensible et respectant les principes de la programmation orientée objet (POO). Notre architecture s'appuie sur le diagramme de classes détaillé de la structure du moteur SGBD :

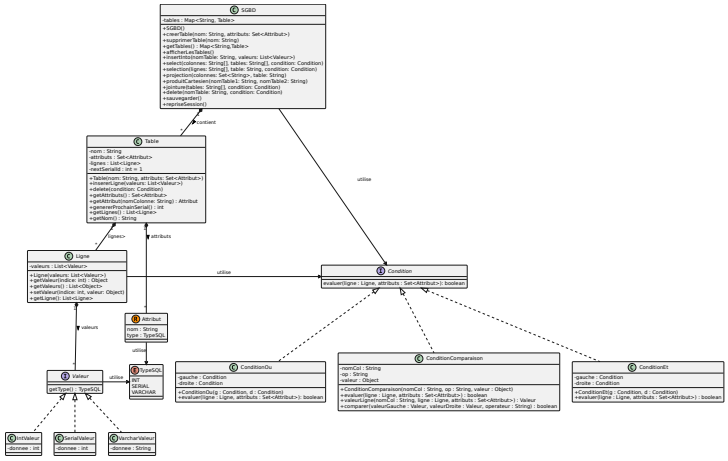


FIGURE 3 – Diagramme de Conception.

2.1 Architecture Globale du SGBD et Encapsulation

Le cœur du système repose sur une hiérarchie de structures imbriquées qui respectent une logique de responsabilité unique :

— **La classe `SGBD`** : Elle agit comme la façade principale du système. Le choix a été fait de centraliser le catalogue des tables au sein d'une structure `Map<String, Table>`. Cette clé/valeur associe le nom d'une table à son instance physique, garantissant un accès en temps constant $O(1)$ lors de la recherche, de la suppression (`supprimerTable`) ou de l'insertion.

— **La gestion du stockage (`Table` et `Ligne`)** : Une `Table` encapsule son schéma (`Set<Attribut>`) et ses données (`List<Ligne>`). Le choix de l'interface `Set` pour les attributs interdit par nature la création de colonnes doublons au sein d'un même schéma. Une `Ligne` est modélisée de manière totalement générique par une liste de conteneurs `Valeur`, ce qui découple la structure de la table du type de données qu'elle transporte.

2.2 Typage Fort et Polymorphisme des Données

Pour gérer les trois types requis par la grammaire (`INT`, `SERIAL`, `VARCHAR`), nous avons mis en place une structure polymorphique stricte afin d'éviter le recours à des objets primitifs Java non contrôlés :

— **L'énumération `TypeSQL`** : Elle liste explicitement les types supportés et sert de référence de vérification lors du `CREATE TABLE`.

— **L'interface `Valeur` et ses implémentations** : Les classes `IntValeur`, `VarcharValeur` et `SerialValeur` implémentent toutes l'interface `Valeur`. Ce choix de conception par polymorphisme permet au SGBD de manipuler des lignes contenant des types

hétérogènes de manière unifiée, tout en conservant la capacité de valider la cohérence du type d'une donnée par rapport au type déclaré dans l'objet `Attribut` associé.

— **Spécificité du type** `SERIAL` : Pour le type auto-incrémenté, la responsabilité de la génération de l'identifiant unique a été placée directement au niveau de la classe `Table` via l'attribut static `nextSerialId` et la méthode `genererProchainSerial()`. Cela garantit qu'aucune entité externe ne peut corrompre la séquence d'indexation d'une table donnée.

2.3 Moteur d'Évaluation Logique

L'évaluation de la clause `WHERE` doit permettre de traiter aussi bien une condition simple, comme `puissance < 10`, qu'une expression plus complexe combinant plusieurs critères à l'aide des opérateurs logiques `AND` et `OR`.

Pour répondre à ce besoin, nous avons mis en place plusieurs classes représentant les différents types de conditions :

— `Condition` : cette interface définit la méthode `evaluer(Ligne, Set<Attribut>)` utilisée pour vérifier si une ligne satisfait une condition.

— `ConditionComparaison` : cette classe représente une condition simple basée sur une comparaison entre une colonne et une valeur, ou entre deux colonnes. Elle prend en charge les opérateurs de comparaison tels que `=`, `>`, `<`, et réalise l'évaluation à l'aide de la méthode `comparer()`.

— `ConditionEt` et `ConditionOu` : ces classes permettent de combiner plusieurs conditions. Elles évaluent les conditions qu'elles contiennent puis appliquent respectivement les opérateurs logiques `AND` et `OR`.

Lors de l'exécution d'une requête, le moteur appelle simplement la méthode `evaluer()` sur la condition principale. Celle-ci se charge ensuite d'évaluer l'ensemble des sous-conditions nécessaires afin de déterminer si la ligne de données respecte les critères définis dans la clause `WHERE`. Cette organisation permet de gérer efficacement aussi bien des conditions simples que des expressions logiques complexes.

2.4 Découplage pour l'Algèbre Relationnelle (SELECT)

La méthode `select` de la classe `SGBD` orchestre l'ensemble des opérations de l'algèbre relationnelle. Pour éviter de surcharger cette méthode, la logique interne a été découpée en trois sous-fonctions privées et spécialisées :

1. `produitCartesien` : Fusionne les tables sources requises si plusieurs éléments sont détectés dans le `FROM`.
2. `selection` : Filtre les lignes du résultat intermédiaire en appliquant l'arbre de `Condition` construit.
3. `projection` : Restreint le tableau final aux seules colonnes demandées dans la clause `SELECT`.

Enfin, la classe `TableSource` regroupe le nom réel de la table ainsi que son éventuel alias.

Cette information est utilisée lors de l'exécution de la requête select afin d'identifier les tables à parcourir et de gérer les alias définis par l'utilisateur. Lorsqu'un alias est présent, le SGBD crée une copie temporaire de la table sous ce nouveau nom afin de pouvoir

manipuler indépendamment plusieurs références à une même table dans une requête. Cette approche simplifie notamment le traitement des jointures et garantit que les noms utilisés dans la requête correspondent correctement aux tables manipulées pendant l'exécution.

3 Implementation

3.1 Implémentation de l'Interpréteur de Commandes SQL

Pour donner vie à notre base de données et exécuter les requêtes de la grammaire *Student Gotham Bat Database*, nous avons développé un interpréteur de commandes interactif en utilisant l'outil **JavaCC**. Le fichier de configuration `InterpreteurCommandeSQL.jj` génère ainsi la classe principale `InterpreteurCommandeSQL`.

Contrairement à un simple analyseur syntaxique passif, notre classe joue le rôle d'**interpréteur direct** : elle valide la structure des commandes SQL saisies par l'utilisateur et déclenche immédiatement les traitements logiques correspondants sur notre moteur `SGBD`.

3.1.1 Configuration globale et analyse lexicale

Dans les options du fichier `.jj`, plusieurs choix stratégiques ont été configurés :

— `IGNORE_CASE = true;` : Option cruciale qui permet d'interpréter indifféremment les mots-clés en majuscules ou en minuscules (`SELECT` équivaut à `select`).

— `SKIP` : Définie pour ignorer les caractères invisibles (espaces, tabulations, retours à la ligne) ainsi que les commentaires SQL introduits par `-`.

Le token d'identification `ID` a été écrit pour tolérer le caractère `.` (`["A"-"Z","a"-"z"] (["A"-"Z","a"-"z","0"-"9","_","."])*`) afin que le parseur sache lire les désignations explicites de colonnes lors des jointures (ex : `E.manager_id`).

3.1.2 Architecture et liaison avec le SGBD

L'interpréteur a besoin d'un accès permanent au moteur de base de données pour appliquer les requêtes lues. Nous avons choisi d'injecter l'instance du `SGBD` directement au moment de l'initialisation de l'interpréteur dans le bloc `PARSER_BEGIN` :

```
public class InterpreteurCommandeSQL {
  private SGBD sgbd;

  public InterpreteurCommandeSQL(SGBD sgbd, java.io.InputStream stream){
    this(stream);
    this.sgbd = sgbd;
  }
}
```

Grâce à cette architecture orientée objet, chaque règle de commande validée se traduit par un appel direct aux méthodes d'exécution du SGBD (comme `sgbd.creerTable()` ou `sgbd.insertInto()`).

3.1.3 Gestion avancée du cycle de vie des requêtes

Le point d'entrée principal : L'axiome `commande()` attend l'une des sept commandes SQL supportées par notre système, immédiatement suivie d'un point-virgule (`<POINT_VIRG>`).

Résolution des alias de tables dans le SELECT : La commande `SELECT` utilise la sous-règle récursive `source()`. Dès que l'interpréteur détecte une table et son mot-clé optionnel `AS`, il instancie un objet `TableSource` contenant le nom réel de la table et l'alias choisi (qui reprend le nom de la table par défaut s'il un alias n'a pas été entré). Ces objets sont accumulés dans un `Set<TableSource> sources` avant d'être envoyés à la méthode `sgbd.select(colonnes, sources, cond)`. Cette approche résout un problème que nous avions rencontré lors des auto-jointures, où une même table physique intervient sous deux rôles différents (par exemple : `FROM Employes AS E, Employes M`).

Initialement, si nous collections uniquement les noms des tables sous forme de chaînes de caractères (`String`), le fait d'avoir deux fois la table `"Employes"` provoquait un doublon. Lors de l'ajout dans un ensemble (`Set`), la deuxième occurrence écrasait la première, empêchant le SGBD de comprendre que la table devait être dupliquée pour la requête.

Pour régler cela, l'utilisation de notre classe `TableSource` permet d'associer de manière unique chaque table à son alias dès l'analyse.

Interprétation des expressions logiques Pour le filtrage des clauses `WHERE` (dans `SELECT` et `DELETE`), l'interpréteur construit un arbre de conditions. La méthode `expressionLogique()` assemble des objets `ConditionComparaison` simples et les imbrique de manière récursive dans des structures composites `ConditionEt` ou `ConditionOu` selon les opérateurs rencontrés. C'est cet arbre abstrait bien structuré qui est transmis au SGBD pour filtrer dynamiquement les lignes lors de l'évaluation.

3.2 Implémentation du Moteur SGBD en Java

Parallèlement à l'interpréteur, nous avons implémenté la logique métier du moteur en Java. Au cours de cette phase, notre conception initiale a évolué afin de simplifier le code et de répondre de façon pragmatique aux problèmes rencontrés lors de la manipulation des données.

3.2.1 Typage fort et interface `Valeur`

Pour gérer les différents types de données de manière sécurisée, nous avons mis en place une énumération `TypeSQL` (regroupant `INT`, `SERIAL` et `VARCHAR`) couplée à une interface `Valeur`. Cette interface est implémentée par les classes `IntValeur`, `SerialValeur` et `VarcharValeur`.

L'intérêt principal de cette structure est de faciliter les opérations de test et de validation des types, tout en permettant d'obtenir leurs valeurs facilement. C'est particulièrement utile dans la méthode `insererLigne()` de la classe `Table` : avant d'ajouter une nouvelle ligne, le système peut comparer le type de chaque valeur du tuple avec le type défini dans les `Attributs` de la table (ces derniers ayant été implémentés sous forme de `record` Java pour alléger le code). Cela garantit l'intégrité des données en empêchant l'insertion de valeurs incohérentes.

3.2.2 Gestion des alias avec `TableSource`

L'implémentation de la commande `SELECT` nous a posé quelques difficultés, en particulier pour faire fonctionner correctement les alias, qui sont indispensables pour réaliser des auto-jointures. Si nous n'avions gardé que le nom des tables sous forme de chaînes de caractères brutes, le système n'aurait pas pu différencier deux appels à la même table au sein d'une même requête.

Pour résoudre ce problème, nous avons intégré la classe `TableSource`, qui stocke à la fois le nom réel de la table et l'alias qui lui est attribué dans la requête. Cela permet au moteur, lors de l'étape de création du produit cartésien, d'isoler et de manipuler plusieurs occurrences d'une même table comme des entités logiques distinctes.

3.2.3 Évaluation des conditions

Enfin, le filtrage des lignes lors des requêtes repose sur l'interface `Condition`. Bien que la structure ait déjà été modélisée en amont, son implémentation en Java s'avère très puissante : elle permet d'évaluer n'importe quelle clause `WHERE` de manière récursive. Un simple appel à la méthode `evaluer()` sur le nœud racine suffit à parcourir l'arbre de conditions, qu'il contienne une simple `ConditionComparaison` ou des combinaisons imbriquées (`ConditionEt`, `ConditionOu`).

Même si ce point a déjà été évoqué auparavant, c'est sûrement le plus gros ajout de conception qui n'avait pas été pensé initialement. Cela nous a fait réaliser qu'entre ce qu'on imagine sur le papier et ce qu'on doit coder en vrai, il y a souvent négligence. Pour nous qui débutons en développement, devoir modifier notre diagramme en plein milieu du projet nous a un peu surpris, mais ça nous a beaucoup appris. Nous avons compris que la conception logicielle n'est pas un cadre figé et qu'il faut savoir s'adapter quand on bloque sur la technique. C'est avec cette expérience pratique en tête que nous pouvons maintenant passer à la conclusion de notre rapport.



FIGURE 4 – Cycle en Spirale.

Conclusion

Ce projet de conception et d'implémentation d'un système de gestion de bases de données relationnelles personnalisé, le *Student Gotham Bat Database*, nous a permis de traverser l'ensemble des couches logicielles qui composent un moteur SQL moderne. En partant d'une feuille blanche, nous avons réussi à bâtir une architecture fonctionnelle répondant aux exigences strictes imposées par le cahier des charges du Chevalier Noir.

Sur le plan technique, ce travail a formalisé nos compétences en ingénierie logicielle et en programmation orientée objet en Java. L'application d'une structure hiérarchique récursive s'est avérée être une solution élégante et hautement évolutive pour résoudre la problématique de l'évaluation des conditions logiques de la clause `WHERE`. De même, la séparation nette entre le moteur de stockage (`SGBD`, `Table`, `Ligne`) et l'interpréteur syntaxique généré via JavaCC garantit un code maintenable et découplé.

Bien que le système soit d'ores et déjà capable d'exécuter des requêtes de base de l'algèbre relationnelle (projections, sélections et produits cartésiens), plusieurs perspectives d'évolution permettraient d'en accroître les performances et la robustesse. À court terme, l'implémentation de clés primaires et étrangères assurerait une meilleure intégrité référentielle des données. Enfin, il serait également pertinent de séparer la logique métier de l'interface utilisateur afin de rendre le système plus clair, plus maintenable et plus facilement évolutif.

Répartition des tâches du projet

Maissa	Yesmina	Ewen	Youssef	Jules
Interpreteur	Interpreteur	Moteur	Moteur	Moteur
Diagramme modélisation	Produit cartésien	SGBD	Lignes	Diagramme de Classe
Condition/Alias	Sélection	Projection	Attributs	Table
Test interpréteur	Exceptions	Select	Tests	Tests
		Affichage		